



PPQ 3

General Note

The only things as required will be submitted as `Q3_bmjm501_main.cpp` and `Q3_bmjm501.pdf`, but a full TAR file called `Q3_bmjm501.tar` will be submitted alongside for a full look at the project, however is not required as this PDF answers all questions. I make references to files in the wider project, but again is not necessary to understand what I have done. (Further note, sorry for the poor formatting, I used markdown to make writing easier but I am unsure on how to convert it into a PDF without bad formatting. It is however all readable.)

What is the value of sum for $N = 10^6$?

As can be seen in `results`, I have consistent results for the sums across a different number of `OMP_NUM_THREADS`

For $N = 10^6$, I got a value of `500002285262.390808`

A preview of the sum data for 12 threads is below:

RUNNING ON 12 THREADS

-----SERIAL-----

N value: 1000 Sum: 502283.876026 Time: 0.000042
N value: 2000 Sum: 2004572.019639 Time: 0.000063
N value: 5000 Sum: 12511473.819668 Time: 0.000156
N value: 10000 Sum: 50022902.063502 Time: 0.000312
N value: 20000 Sum: 200045747.493330 Time: 0.001106
N value: 50000 Sum: 1250114373.269153 Time: 0.003593
N value: 100000 Sum: 5000228702.507833 Time: 0.007949
N value: 200000 Sum: 20000457210.329525 Time: 0.015892
N value: 500000 Sum: 125001142626.868362 Time: 0.041084
N value: 1000000 Sum: 500002285262.390808 Time: 0.082211
N value: 2000000 Sum: 2000004569634.123291 Time: 0.165427
N value: 5000000 Sum: 12500011423817.324219 Time: 0.415646

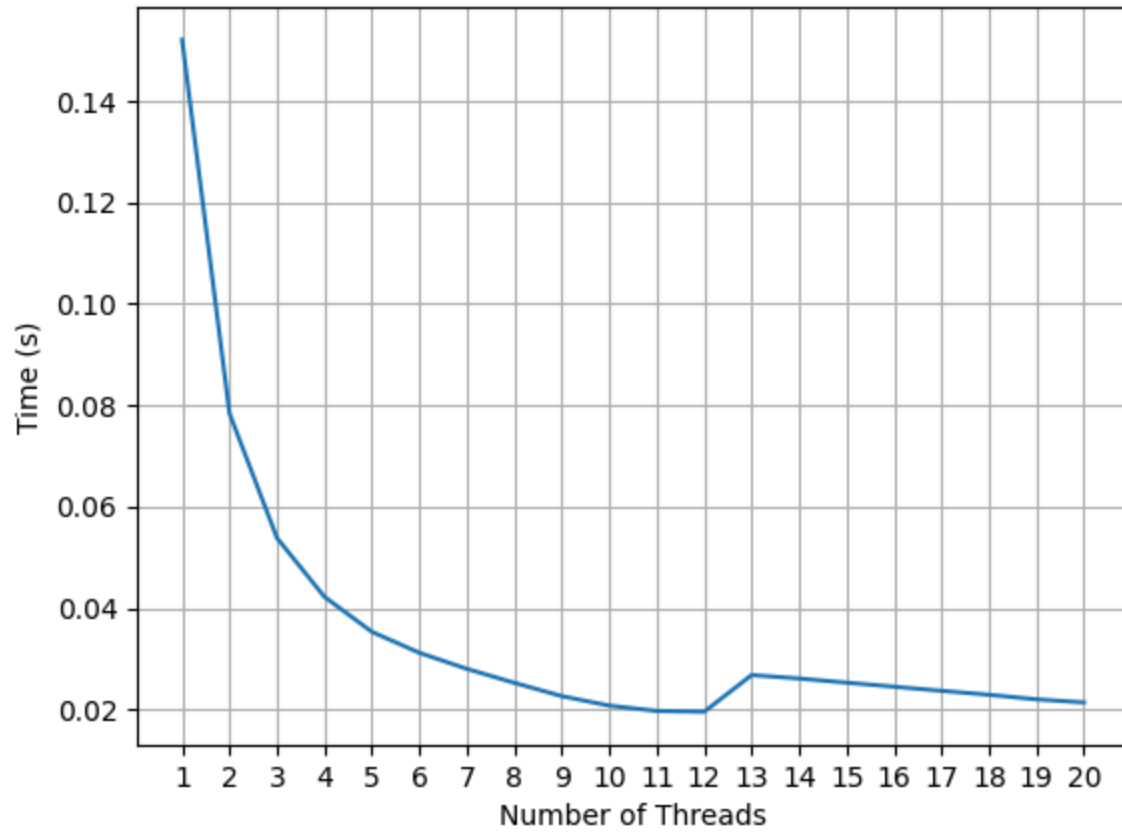
-----PARALLEL-----

N value: 1000 Sum: 502283.876026 Time: 0.001690
N value: 2000 Sum: 2004572.019639 Time: 0.000013
N value: 5000 Sum: 12511473.819668 Time: 0.000025
N value: 10000 Sum: 50022902.063502 Time: 0.000048
N value: 20000 Sum: 200045747.493330 Time: 0.000181
N value: 50000 Sum: 1250114373.269153 Time: 0.000454
N value: 100000 Sum: 5000228702.507833 Time: 0.000934
N value: 200000 Sum: 20000457210.329525 Time: 0.001939
N value: 500000 Sum: 125001142626.868362 Time: 0.007821
N value: 1000000 Sum: 500002285262.390808 Time: 0.011539
N value: 2000000 Sum: 2000004569634.123291 Time: 0.021158
N value: 5000000 Sum: 12500011423817.324219 Time: 0.055679

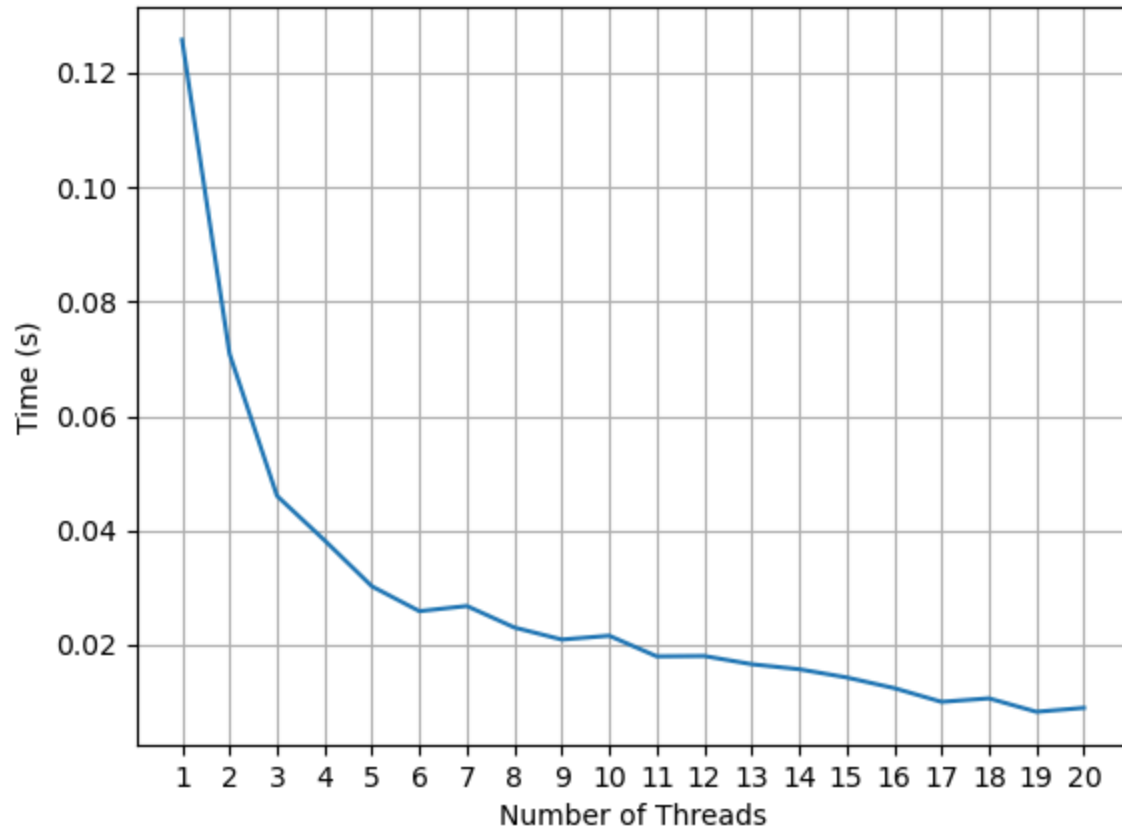
Parallel Speedup

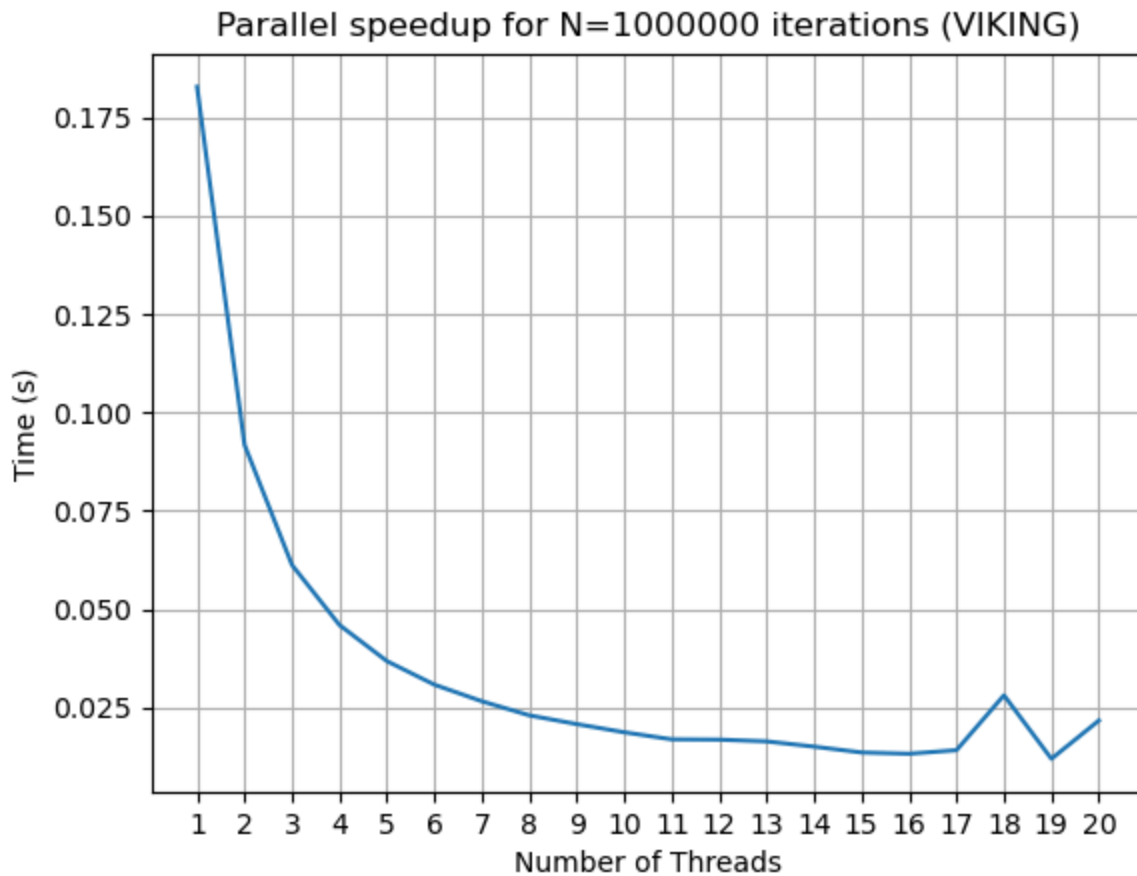
The graphs below are the parallel speedup from 1 to 20 threads on three different machines. My Home PC (6 cores), Teaching0 and VIKING.

Parallel speedup for N=1000000 iterations (HomePC)



Parallel speedup for N=1000000 iterations (Teaching0)





In general, we can see across the machines we are getting exponential speedup as the number of threads increase. At the lower thread counts, the decrease is more rapid as each new thread can be assigned to it's own core, which may have hyperthreading built into it. However as the number of threads equals the hardware limit, we start to see diminishing returns. Past a certain point, we get small bumps which is likely due to background noise on the machine, particularly for when I am exceeding the amount of cores on the Home PC. Further Teaching0 had a lot of noise, but a rough exponential shape is formed. Note these times per thread was the average of 100 runs of the function. This can be seen in `src/Q3_bmjm501_main.cpp: analysis()` or below in the code listing.

Specifically for the Home PC, there is a $\sim 1.94x$ speedup between 1 to 2 threads, and $\sim 1.46x$ speedup between 2 to 3 threads.

Code listing

The main source file can be found in `src/Q3_bmjm501_main.cpp`, however a listing

with explaining comments is below:

```

#include "vtimer_t.h"
#include <cmath>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <omp.h>
#include <vector>
#include <fstream>

using namespace std;

// Apply Kahan Summation algorithm for a vector of values
double kahan_sum(const vector<double> &input)
{
    double sum = 0.0;
    double compensation = 0.0;

    for (double num : input)
    {
        double y = num - compensation;
        double t = sum + y;
        compensation = (t - sum) - y;
        sum = t;
    }

    return sum;
}

// Perform algorithm and output the results without any parallisation
void serial(vector<int> &N_values)
{
    vtimer_t timer; // Start timer
    for (int j = 0; j < N_values.size(); ++j)
    {
        int N = N_values[j];
        vector<double> A(N, 0.0), B(N, 0.0); // Allocate vectors
        timer.start();
        double sum = 0.0;
    }
}

```

```

double compensation = 0.0;

for (int i = N - 1; i >= 0; i -= 1)
{
    A[i] = i + 1; // Perform the algorithm
    B[i] = A[i] * A[i];
    // A[i] = A[i] + sqrt(abs(sin(B[i]))) * 2.34;

    /*
       Instead of doing sum += A[i] we apply the Kahan summation algorithm
       for more accurate sums when N is very large
    */
    double y = sqrt(abs(sin(B[i]))) * 2.34 - compensation;
    double t = sum + y;
    compensation = (t - sum) - y;
    sum = t;
}
sum = sum + 0.5 * N * (N + 1);
timer.stop(); // End timer
string message = "N value: " + to_string(N) +
                " Sum: " + to_string(sum) +
                " Time: " + to_string(timer.elapsed_time());
printf("%s \n", message.c_str());
}
}

// Perform algorithm and output results with parallisation
void parallel(vector<int> &N_values)
{
    for (int j = 0; j < N_values.size(); ++j)
    {
        int N = N_values[j];
        vector<double> A(N, 0.0), B(N, 0.0); // Allocate vectors A and B
        double start_time = omp_get_wtime(); // Start parallel timer

        /* Allocate vectors for each thread sum*/
        vector<double> thread_sums(omp_get_max_threads(), 0.0);
        vector<double> thread_compensations(omp_get_max_threads(), 0.0);
    }
}

```

```

// Go parallel
#pragma omp parallel default(none) \
    shared(A, B, N, thread_sums, thread_compensations)
{
    // Get the thread number, and initialise a local sum and compensation for KS a
    int tid = omp_get_thread_num();
    double local_sum = 0.0;
    double local_compensation = 0.0;

#pragma omp for
    for (int i = N - 1; i >= 0; i -= 1)
    {
        // Apply algorithm
        A[i] = i + 1;
        B[i] = A[i] * A[i];
        // A[i] = A[i] + sqrt(abs(sin(B[i]))) * 2.34;

        /*
        Instead of doing sum += A[i] we apply the Kahan summation algorithm
        for more accurate sums when N is very large
        */
        double y = sqrt(abs(sin(B[i]))) * 2.34 - local_compensation;
        double t = local_sum + y;
        local_compensation = (t - local_sum) - y;
        local_sum = t;
    }

    // No race conditions as each thread has a unique id
    thread_sums[tid] = local_sum;
    thread_compensations[tid] = local_compensation;
}

// Instead of reduction, kahan sum the array
double total_sum = kahan_sum(thread_sums);
total_sum = total_sum + 0.5 * N * (N + 1);

double end_time = omp_get_wtime();

```

```

        printf("N value: %d Sum: %f Time: %f\n", N, total_sum,
               end_time - start_time);
    }
}

// Similar function to above, but for only one N value
double parallel_for_mean(int &N)
{
    vector<double> A(N, 0.0), B(N, 0.0);
    double start_time = omp_get_wtime();

    vector<double> thread_sums(omp_get_max_threads(), 0.0);
    vector<double> thread_compensations(omp_get_max_threads(), 0.0);

#pragma omp parallel default(none) \
    shared(A, B, N, thread_sums, thread_compensations)
    {
        int tid = omp_get_thread_num();
        double local_sum = 0.0;
        double local_compensation = 0.0;

#pragma omp for
        for (int i = N - 1; i >= 0; i -= 1)
        {
            // Apply algorithm
            A[i] = i + 1;
            B[i] = A[i] * A[i];
            // A[i] = A[i] + sqrt(abs(sin(B[i]))) * 2.34;

            /*
            Instead of doing sum += A[i] we apply the Kahan summation algorithm
            for more accurate sums when N is very large
            */
            double y = sqrt(abs(sin(B[i]))) * 2.34 - local_compensation;
            double t = local_sum + y;
            local_compensation = (t - local_sum) - y;
            local_sum = t;
        }
    }
}

```

```

        thread_sums[tid] = local_sum;
        thread_compensations[tid] = local_compensation;
    }

    double total_sum = kahan_sum(thread_sums);
    total_sum = total_sum + 0.5 * N * (N + 1);

    double end_time = omp_get_wtime();

    return end_time - start_time;
}

// Function to run the parallel algorithm and average the time
void analysis()
{
    int N = 1000000;
    vector<double> times;
    double time_sum = 0.0;

    for (int i = 0; i < 100; i++)
    {
        times.push_back(parallel_for_mean(N));
    }

    for (double num : times)
    {
        time_sum += num;
    }

    ofstream file;
    string file_path = "./results/parra_times_XT.txt";
    file.open(file_path, ios::app);

    file << (time_sum / times.size()) << endl;
    file.close();
}

```

```

/* This main is for getting results for all N values*/
int main()
{
    vector<int> N_values = {1000, 2000, 5000, 10000, 20000, 50000,
                           100000, 200000, 500000, 1000000, 2000000, 5000000};

    cout << "RUNNING ON " << omp_get_max_threads() << " THREADS" << endl;
    cout << "-----SERIAL-----" << endl;
    serial(N_values);
    cout << "-----PARALLEL-----" << endl;
    parallel(N_values);
    return 0;
}

/* This main is for analysis*/
// int main()
// {
//     analysis();
// }

```

With other scripts like `analysis/graph.py`, `bin/get_times.sh` and `ppq3_viking.sh` used for graphing, doing shell commands to change the number of threads and running on VIKING respectively.

It is important to note a modification that I made to the general algorithm. Instead of doing:

```

A[i] = A[i] + sqrt(abs(sin(B[i]))) * 2.34;
double y = A[i] - local_compensation;
double t = local_sum + y;
local_compensation = (t - local_sum) - y;
local_sum = t;

```

i.e. summing both the integer part `A[i]` and the smaller double part given by the `sqrt` expression, the resulting sum for $N = 10^6$ was not consistent over different numbers of threads. To fix this, I instead implemented:

```
double y = sqrt(abs(sin(B[i]))) * 2.34 - local_compensation;
double t = local_sum + y;
local_compensation = (t - local_sum) - y;
local_sum = t;
// more code
double total_sum = kahan_sum(thread_sums);
total_sum = total_sum + 0.5 * N * (N + 1);
```

i.e only apply the Kahan summation algorithm to the small double part then add on the integer sum value using the expression $0.5N(N + 1)$. This of course is mathematically the same and this fix then gave consistent results over all threads. Notably however, the only non-matching sums before the fix were due to the $N = 1000000$ and $N = 2000000$ term, for which I do not have an explanation for.